

Student Notes Manager

Project Introduction & Database Design

From individual Flask concepts to a real multi-feature application

Requirements

User Flow

ER Design

SQL Schema

Project Milestone

Trainer

Madhu Parvathaneni, CTO - Codegnan
madhu@codegnan.com - 8121289993

Today's learning map

Day 6 milestone: design the Student Notes Manager before writing major features.



Project idea

What problem are we solving?

Requirements

Functional vs non-functional

User flow

How the student moves through the app

Database plan

Entities, attributes & relationships

SQL schema

Users, notes and file data

Practice

Create, test and explain the schema



Class outcome: students should leave with a working database schema and a clear explanation of why every table and relationship exists.

Quick recall from Day 5

Before database design, connect user input to the data we plan to store.



```
1 name = request.form.get('name')
2 email = request.form.get('email')
3 message = request.form.get('message')
```

Question 1

Where should persistent user data live?

Question 2

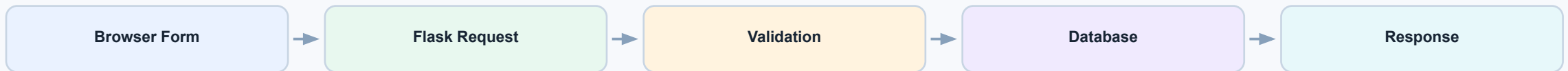
What happens if the server restarts?

Question 3

How do we connect a note to its owner?

Question 4

Which values must be unique?



Transition question: request.form gives us data for one request; a database lets the application remember data across requests and restarts.

Project: Student Notes Manager

A single project that will grow across Days 6-20.



User account

Register, verify, login, reset password

Notes

Create, read, update, delete and search

Files

Upload, view, download and delete

Security

Hash passwords, protect ownership, validate input

APIs

Expose selected functionality using REST

Production thinking

Configuration, logging, testing and deployment



Day 6 is design day: if we design the data model badly now, every later feature becomes harder.

Requirements first, code second

Requirements tell us what the system must do and how well it must behave.



Functional requirements

Visible capabilities: register, login, create notes, upload files, search notes, logout, reset password.

Non-functional requirements

Quality expectations: security, usability, response time, reliability, maintainability and compatibility.

Example

Functional

“The user can log in with email and password.”

Non-functional

“Invalid login must not reveal whether an email exists.”

Exam/interview rule: functional = WHAT the system does; non-functional = HOW WELL / HOW SAFELY it does it.

Class activity: classify the requirement

Ask students to vote F (functional) or NF (non-functional).



1. User can create a note

F

2. Passwords are never stored as plain text

NF

3. User can search notes

F

4. Pages should remain understandable on mobile

NF

5. User can upload a PDF

F

6. Only the owner can access a private note

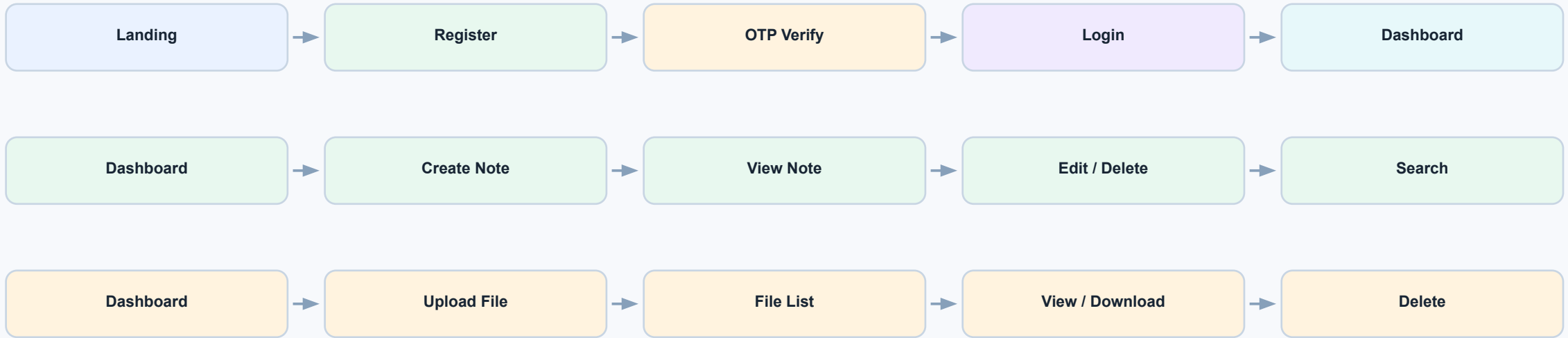
NF

Trainer move: hide the F/NF labels verbally, ask for reasoning, then reveal the answer.

Design the user flow



A user flow shows the sequence of screens/actions a user follows to achieve a goal.



Question

What should happen if the user is not logged in?

Question

What if OTP is expired?

Question

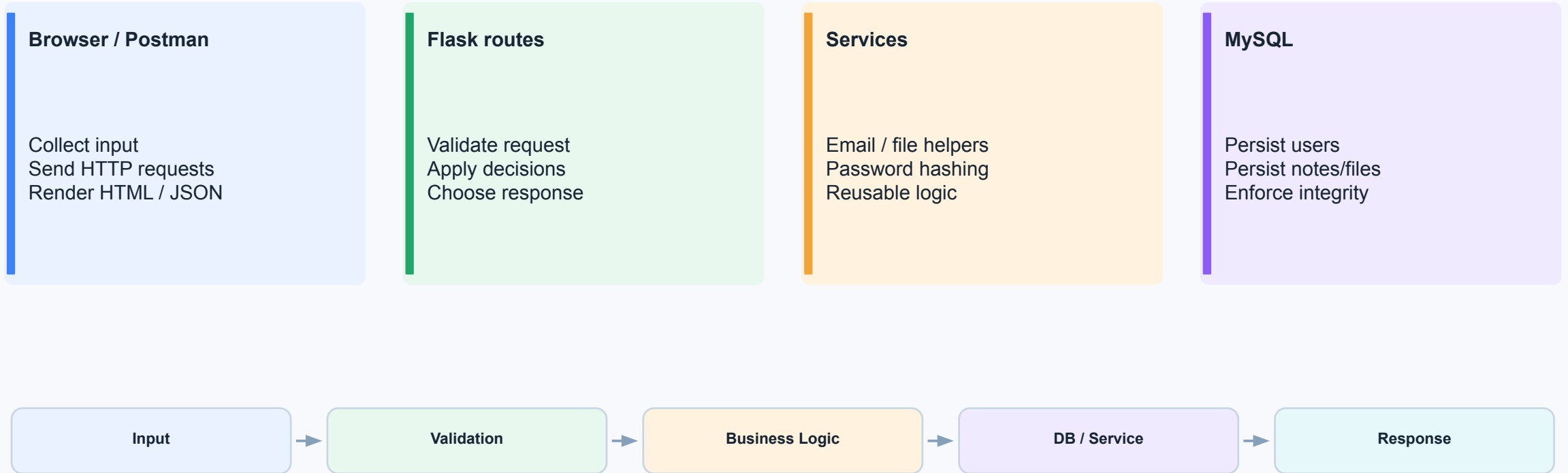
What if a user requests another user's note?

Good design includes failure paths, not only the happy path.

Project workflow: who is responsible for what?



Separate browser, Flask, services and database responsibilities.



Design goal: each layer should have a clear job. This makes debugging easier because we know where to look.

Database planning starts with nouns

Convert project requirements into entities and attributes.



Entity 1: User

id
name
email
password_hash
is_verified

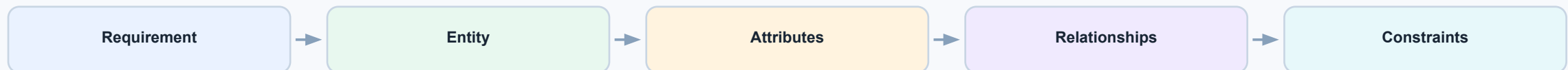
Entity 2: Note

id
user_id
title
content
created_at

Entity 3: FileData

id
user_id
original_name
stored_name
size_bytes

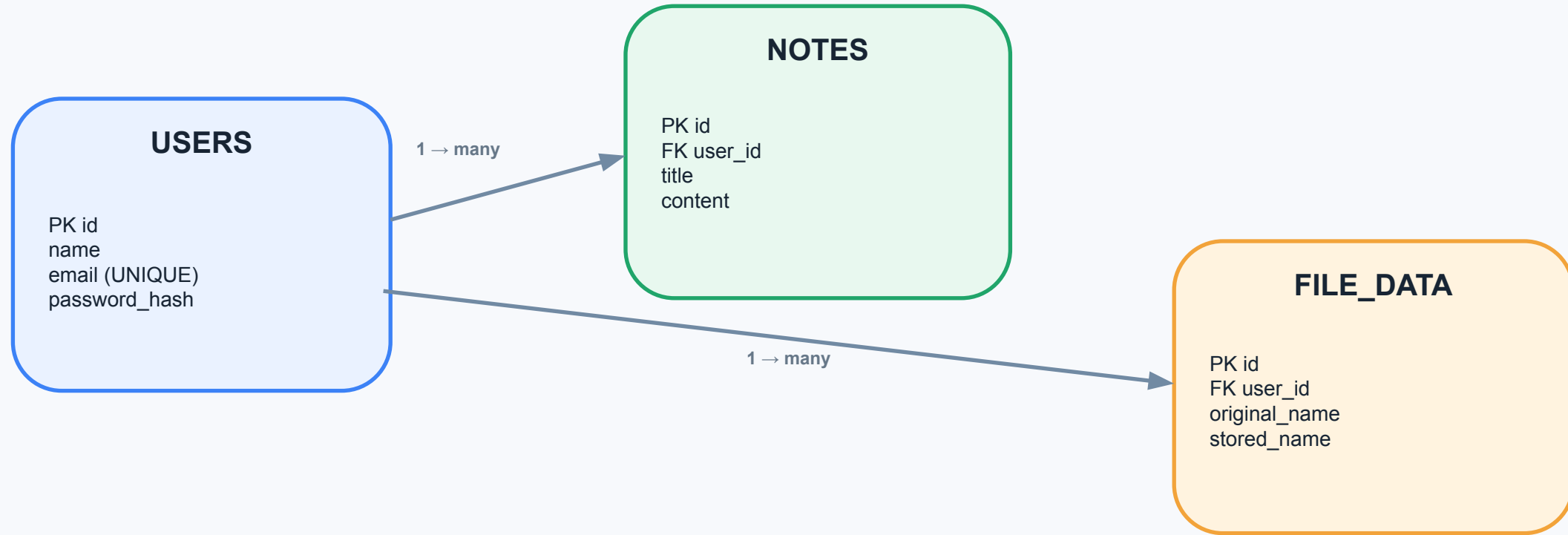
Requirement → noun → entity/table → attributes/columns



Avoid creating tables first and “figuring out relationships later.” Model the domain before writing SQL.

Entity relationships

One user owns many notes and many uploaded files.



Ownership lives in the foreign key: notes.user_id and file_data.user_id point to users.id.

Primary key: the stable identity of a row

A primary key must uniquely identify one record.



Good choice

`id BIGINT AUTO_INCREMENT`

Poor choice

name - duplicates are possible

Risky relationship key

email - unique, but business values may change

```
1 id BIGINT UNSIGNED
2 AUTO_INCREMENT PRIMARY KEY
```

Unique

Stable

Never NULL

Referenced by FKs

Memory rule: primary key answers "Which exact row?"

Foreign key: the relationship guard

A foreign key prevents a child record from pointing to a parent that does not exist.



```
1 user_id BIGINT UNSIGNED NOT NULL,  
2 CONSTRAINT fk_notes_user  
3   FOREIGN KEY (user_id)  
4   REFERENCES users(id)  
5   ON DELETE CASCADE
```

Valid insert

user_id = 5 and user 5 exists

Invalid insert

user_id = 999 but user 999 does not exist

ON DELETE CASCADE

Deleting a user automatically deletes dependent notes/files. Use intentionally.



Memory rule: primary key identifies; foreign key connects.

Constraints protect data integrity

Validation in Flask improves UX; constraints protect the database from every access path.



NOT NULL

Value is required

UNIQUE

No duplicates

FOREIGN KEY

Valid relationship

CHECK

Allowed rule / range

Data type

Correct storage shape

DEFAULT

Reliable starting value

```
1 email VARCHAR(255) NOT NULL UNIQUE,  
2 is_verified BOOLEAN NOT NULL DEFAULT FALSE
```

Ask: Which rules should exist only in Python? Which ones deserve database enforcement too?

Designing the users table

Store identity, authentication state and verification metadata.



Why password_hash?

Never design a system that stores plain-text passwords.

Why UNIQUE email?

One account per email address.

Why is_verified?

OTP/email verification needs a persistent state.

Why timestamps?

Useful for auditing and sorting.

```
1 CREATE TABLE users (  
2   id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
3   name VARCHAR(100) NOT NULL,  
4   email VARCHAR(255) NOT NULL UNIQUE,  
5   password_hash VARCHAR(255) NOT NULL,  
6   is_verified BOOLEAN NOT NULL DEFAULT FALSE,  
7   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
8 );
```

Designing the notes table

Every note belongs to exactly one user.



```
1 CREATE TABLE notes (  
2   id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
3   user_id BIGINT UNSIGNED NOT NULL,  
4   title VARCHAR(150) NOT NULL,  
5   content TEXT NOT NULL,  
6   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
7   updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
8     ON UPDATE CURRENT_TIMESTAMP,  
9   FOREIGN KEY (user_id) REFERENCES users(id)  
10     ON DELETE CASCADE  
11 );
```

Ownership

user_id tells us who owns the note.

Content type

TEXT is suitable for longer note content.

updated_at

Automatically tracks last edit time.

Delete behavior

CASCADE removes orphaned notes.

Designing the file_data table

Store file metadata separately from the physical file itself.



```
1 CREATE TABLE file_data (  
2   id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
3   user_id BIGINT UNSIGNED NOT NULL,  
4   original_name VARCHAR(255) NOT NULL,  
5   stored_name VARCHAR(255) NOT NULL,  
6   mime_type VARCHAR(120) NOT NULL,  
7   size_bytes BIGINT UNSIGNED NOT NULL,  
8   storage_path VARCHAR(500),  
9   FOREIGN KEY (user_id) REFERENCES users(id)  
10  ON DELETE CASCADE  
11 );
```

original_name

Name shown back to the user.

stored_name

Server-safe unique filename.

mime_type / size

Useful for validation and response headers.

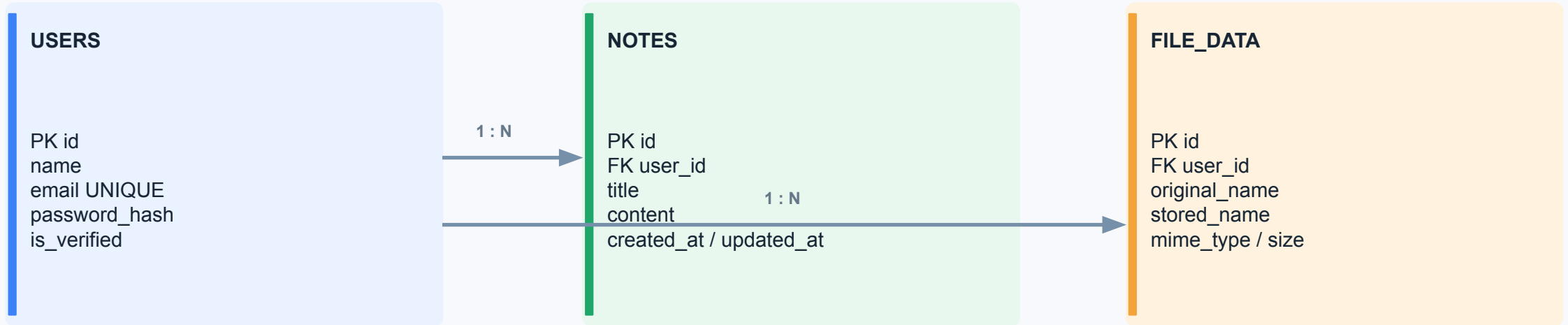
storage_path

Where the file is actually stored.

Complete database picture



Three tables are enough for the first project milestones.



Next days will add routes and UI on top of this data model; the schema becomes the backbone of the project.

Live demo: create the database

Create the schema with Unicode-friendly settings.



```
1 CREATE DATABASE student_notes_manager
2 CHARACTER SET utf8mb4
3 COLLATE utf8mb4_unicode_ci;
4
5 USE student_notes_manager;
```

Step 1

Run CREATE DATABASE.

Step 2

Select it with USE.

Step 3

Verify with SHOW DATABASES.



Live demo: insert sample data

Seed a small data set so we can verify relationships.



```
1 INSERT INTO users(name,email,password_hash,is_verified)
2 VALUES ('Asha','asha@example.com','demo_hash',TRUE);
3
4 INSERT INTO notes(user_id,title,content)
5 VALUES (1,'Flask Day 6','Database design completed');
6
7 SELECT * FROM users;
8 SELECT * FROM notes;
```

Check 1

Did the user receive an id?

Check 2

Does note.user_id match users.id?

Check 3

Try user_id = 999. What happens?

Intentional failure is part of the demo: show that the foreign key protects the database.

Common database design mistakes to catch early



Using email as every relationship key

Use stable numeric ids for relationships.

Choose proper data types

VARCHAR for bounded text, TEXT for long content.

No foreign keys

The DB cannot protect ownership relationships.

Use constraints

NOT NULL, UNIQUE, FK, DEFAULT.

Storing plain passwords

Store only password hashes.

Add timestamps

Useful for sorting and audit history.

No UNIQUE on email

Duplicate accounts become possible.

Test invalid inserts

Verify the database rejects bad states.

Debugging habit: when a query fails, read the exact database error before changing code.

Classroom practice plan

Students design and create the database in small checkpoints.



Round 1 - 8 min

Write functional + non-functional requirements

Round 2 - 10 min

Draw user flow and ER diagram

Round 3 - 15 min

Create users, notes and file_data tables

Round 4 - 10 min

Insert valid sample data

Round 5 - 8 min

Trigger one constraint failure intentionally

Round 6 - 5 min

Explain PK, FK and ownership to a partner

Assessment evidence: schema screenshot + one valid insert + one failed FK/UNIQUE insert + student explanation.

Day 6 quick quiz

Use as an exit ticket or oral recap.



1. Functional vs non-functional requirement?
2. What is an entity?
3. What does a primary key do?
4. What does a foreign key do?
5. Why is email UNIQUE but not our primary key?
6. Why does notes need user_id?
7. What does ON DELETE CASCADE do?
8. Name three useful database constraints.

Answer pattern

Define → explain purpose → give project example

Most important

PK identifies; FK connects.

Project connection

Every note/file belongs to a user.

Ask students to answer without reading SQL. Understanding the design matters more than memorizing syntax.

Homework assignment

Complete the Day 6 milestone independently.



Task 1

Write 8 functional + 5 non-functional requirements

Task 2

Draw the complete user flow

Task 3

Draw ER diagram with cardinalities

Task 4

Create database and three tables

Task 5

Insert at least 2 users and 4 notes

Task 6

Document one failed constraint test

Git commit suggestion: "Day 6: project introduction and database design".

Day 6 recap

The project now has a plan and a data backbone.



1. Start with requirements before coding.
2. Functional requirements describe capabilities.
3. Non-functional requirements describe quality and safety.
4. User flow includes both success and failure paths.
5. Entities become tables; attributes become columns.
6. Primary keys uniquely identify rows.
7. Foreign keys model ownership and protect relationships.
8. Constraints keep invalid states out of the database.

Today

Project + Database Design

Milestone

3-table schema ready

Next class

Project Initialization & UI Design

Build slowly. Design clearly. Test deliberately.